

IFC-Diff Viewer

Was hat sich seit dem letzten Modellstand geändert?

Zwischenpräsentation 1 – Konzept Semesterprojekt
Modul TA.BA_DT_PROGR Digital Twin Programmieren, HS26
Gian Blaser · Hochschule Luzern, Digital Construction
github.com/GianBlaser/HSLU_GB_26_V2

ADDED

DELETED

MODIFIED

UNCHANGED

Ausgangslage: Modellstände vergleichen ist Handarbeit

- In der BIM-Koordination kommen laufend neue IFC-Stände: Architektur, Tragwerk, HLKS – wöchentlich, manchmal täglich.
- Die Frage «Was ist neu, was fehlt, was wurde verschoben?» wird heute meist per Sichtvergleich im Viewer oder per Excel-Export beantwortet.
- Fehleranfällig, nicht reproduzierbar, nicht dokumentiert.
- Kommerzielle Tools (Solibri, BIMcollab) können das – aber als Blackbox und kostenpflichtig.

1 : 1

Element für Element über die GlobalId –
nicht «ungefähr hinschauen»

*Kontext gemäss Merkblatt:
Datenauswertung + Daten-Visualisierung*

Zielsetzung und Funktionsumfang

Ein Python-Programm, das zwei IFC-Stände desselben Projekts vergleicht und die Unterschiede tabellarisch, statistisch und im 3D-Viewer farbcodiert zeigt.

1

Laden

Zwei IFC-Dateien: Stand A (alt), Stand B (neu) – Auswahl oder Upload

2

Vergleichen

Matching über GlobalId → ADDED / DELETED / MODIFIED / UNCHANGED

3

Speichern

Jeder Lauf in SQLite: Historie aller Vergleiche

4

Auswerten

Pandas: Änderungen je Geschoss, je IfcClass, je Änderungsart

5

Visualisieren

Matplotlib-Diagramme; 3D-Viewer: geändert opak farbig, Kontext transparent

6

Exportieren

CSV heute, PDF-Report geplant

Kernlogik: Was gilt als «geändert»?

Eigene Diff-Definition – Schlüssel ist die GlobalId, Matching per Dictionary statt verschachtelter Schleifen.

ADDED

GlobalId nur in Stand B

DELETED

GlobalId nur in Stand A

MODIFIED

in beiden, mind. eine Ebene weicht ab

UNCHANGED

in beiden, alle Ebenen gleich

Drei Vergleichsebenen je Element

1 Direktattribute

Name, Description, ObjectType, Tag, PredefinedType – exakter Vergleich

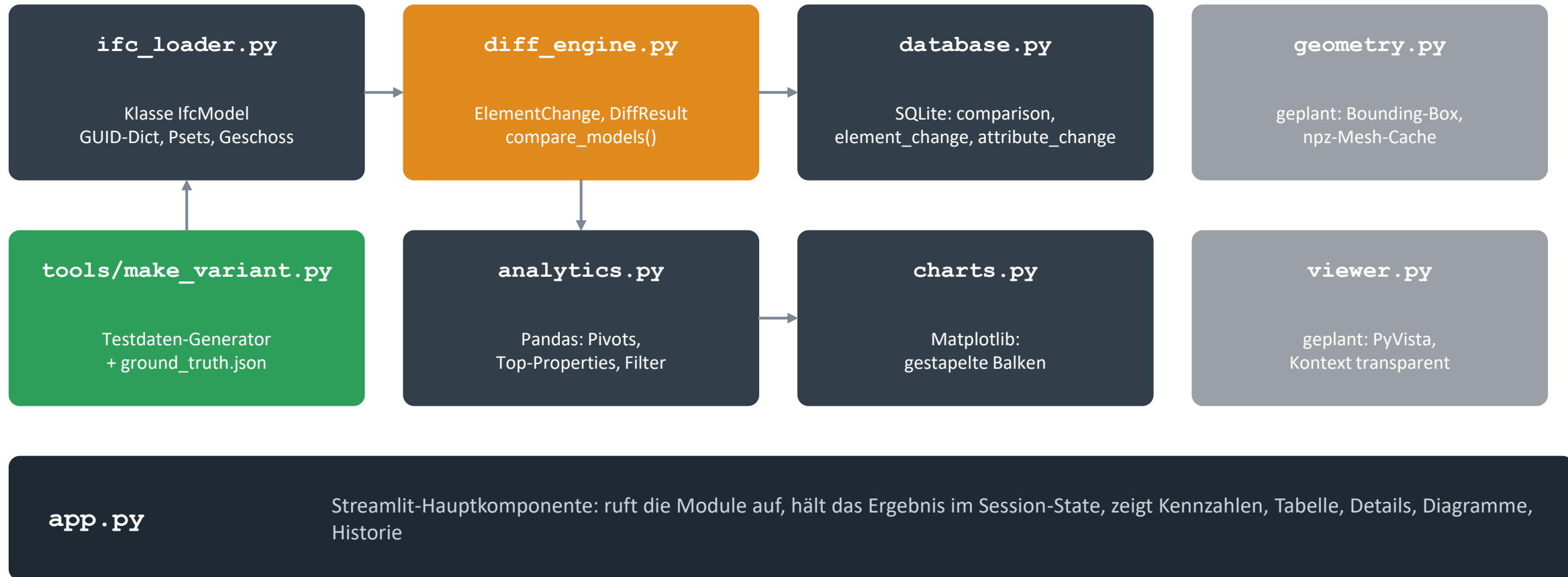
2 Property Sets

get_psets() flach als «Pset.Property». Zahlen mit Toleranz 1e-6, nicht mit ==. Entity-IDs und OwnerHistory ausgeschlossen.

3 Geometrie (vereinfacht)

Heute: Einfügepunkt (Toleranz 1 mm). Geplant: Bounding-Box-Schwerpunkt, Abmessungen, Dreiecksanzahl. Bewusst kein Mesh-Topologie-Vergleich.

Architektur: modularer Aufbau



src/config.py: Farben, Toleranzen, Pfade – keine Magic Numbers im Code. tests/: 20 pytest-Tests, davon 7 gegen die Ground Truth des Generators.

Entwicklungsumgebung, Module und Werkzeuge

Bereich	Werkzeug	Einsatz im Projekt
Sprache	Python 3.13	einzige Entwicklungssprache; venv «myenv» im Repo-Root
Editor	Visual Studio Code	Python-Extension, integriertes Terminal, Git-Ansicht
Versionskontrolle	Git + GitHub (public)	Commit je Etappe, Konvention «[E2] ...», Tag v1.0-abgabefähig
IFC	IfcOpenShell 0.8.5	Pflicht-Modul: Lesen, Psets, Container, Geometrie, API für Testdaten
Daten	Pandas 3.0	Vergleichstabelle, Pivots (crosstab), Filter
Diagramme	Matplotlib 3.11	gestapelte Balken je Geschoss / IfcClass, Top-Properties
Datenbank	SQLite (Standardbibliothek)	Historie aller Vergleichsläufe, 3 Tabellen
UI	Streamlit 1.64	Upload, Filter, Tabelle mit Zeilenauswahl, Tabs
Tests	pytest	20 Tests, Diff-Engine gegen Ground Truth
3D (geplant)	PyVista + stpyvista	ausserhalb der Modulliste – Abklärung heute

Testdaten: Generator statt Zufall

Problem

Es gibt keine öffentlichen IFC-Datensätze mit zwei echten Planungsständen. Die buildingSMART-Testdateien liegen nur in verschiedenen Schema-Versionen vor – ein Diff darauf zeigt Schema-Unterschiede, keine Planungsänderungen.

Lösung

tools/make_variant.py erzeugt aus Stand A reproduzierbar (Seed) einen Stand B: n Elemente löschen, n verschieben, n umbenennen und Mengenwert ändern, n duplizieren mit neuer GlobalId. Alle gemachten Änderungen landen in ground_truth.json.

Effekt

Die Diff-Engine ist automatisiert testbar: jede gesetzte Änderung muss gefunden werden, Datei gegen sich selbst muss 0 ergeben. Eigenleistung, die sich beweisen lässt.

```
ground_truth.json (Seed 42)
```

```
"deleted": 2
  IfcWall "house - outer wall ..."
  IfcBuildingElementProxy "origin"
```

```
"moved": 2
  IfcSlab (2100, 0) → (3100, 500)
  IfcChimney +1000 / +500 mm
```

```
"modified": 2
  Name "... (rev B)"
  Qto NetVolume 4.23 → 4.65
```

```
"added": 2
  Kopie mit neuer GlobalId
```

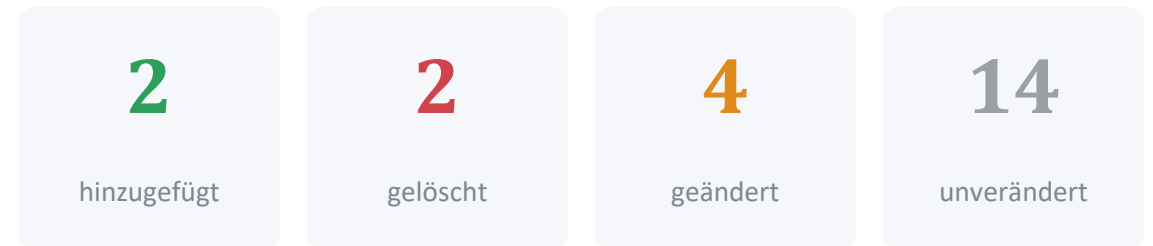
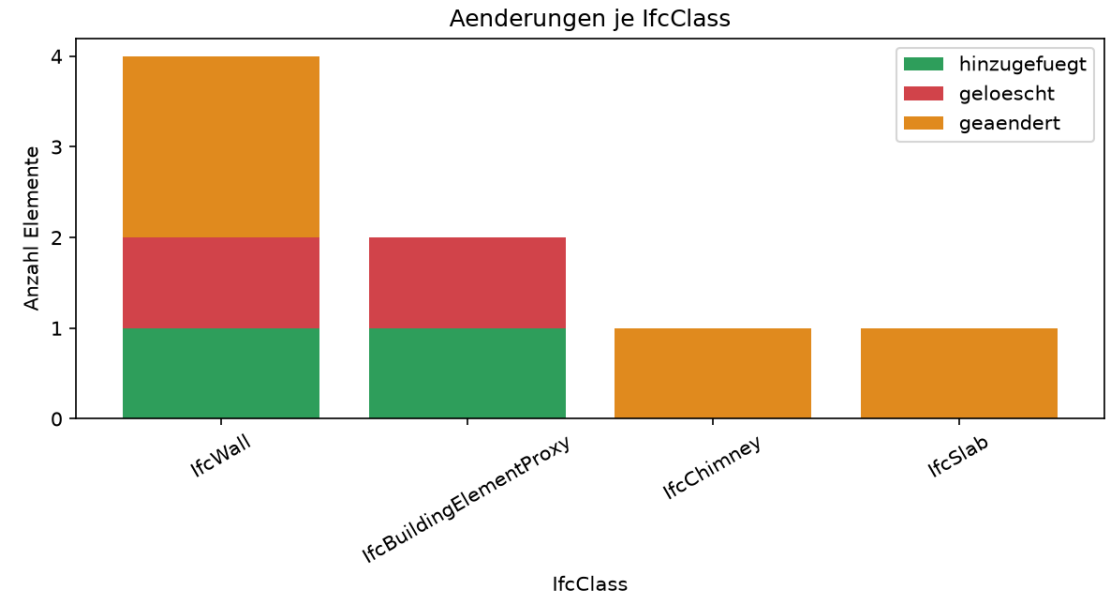
```
→ Diff-Engine findet 2 / 2 / 4 / 14
```

```
→ A gegen A: 0 / 0 / 0 / 20
```

Stand heute: Kette läuft durchgehend

- Etappen 0–5 des Projektplans abgeschlossen (Setup, Testdaten, Diff-Engine, SQLite, Auswertung, Streamlit-UI).
- Benutzbare Version: Dateien wählen → Vergleichen → Kennzahlen, filterbare Tabelle, Detailansicht alt/neu, Diagramme, CSV, Historie.
- 20 automatisierte Tests grün.
- Git-Tag v1.0-abgabefähig – ab hier ist jede Etappe ein Zusatz, kein Risiko.

- E0 Geruest + Setup
- E1 Testdaten-Generator
- E2 Loader + Diff-Engine
- E3 SQLite-Historie
- E4 Auswertung + Diagramme
- E5 Streamlit-UI
- E6 Geometrie + Cache
- E7 3D-Viewer
- E8 Report (optional)



Testlauf Building-Architecture.ifc (IFC4, 20 IfcProduct) gegen generierten Stand B

Herausforderungen und Abgrenzungen

3D-Viewer in Streamlit

PyVista/stpyvista liegt ausserhalb der Modulliste und ist in Streamlit heikel. Fallback: PyVista im eigenen Fenster, Screenshot in der UI. Abklärung heute.

Geometrie-Performance

Tessellierung grosser Modelle dauert.
Massnahmen: ifcopenshell.geom.iterator mit Threads, npz-Cache je Datei-Hash, «nur Änderungen rendern» ab 20 000 Elementen.

Bewusste Abgrenzung

Kein Mesh-Topologie-Vergleich: Geometrie nur über Einfügepunkt, Bounding-Box und Dreiecksanzahl. Nur gleiche IFC-Schemata; sonst Warnung.

Testbarkeit gelöst

Fehlende Zwei-Stände-Testdaten durch eigenen Generator mit Ground Truth ersetzt – Diff-Engine ist beweisbar korrekt.

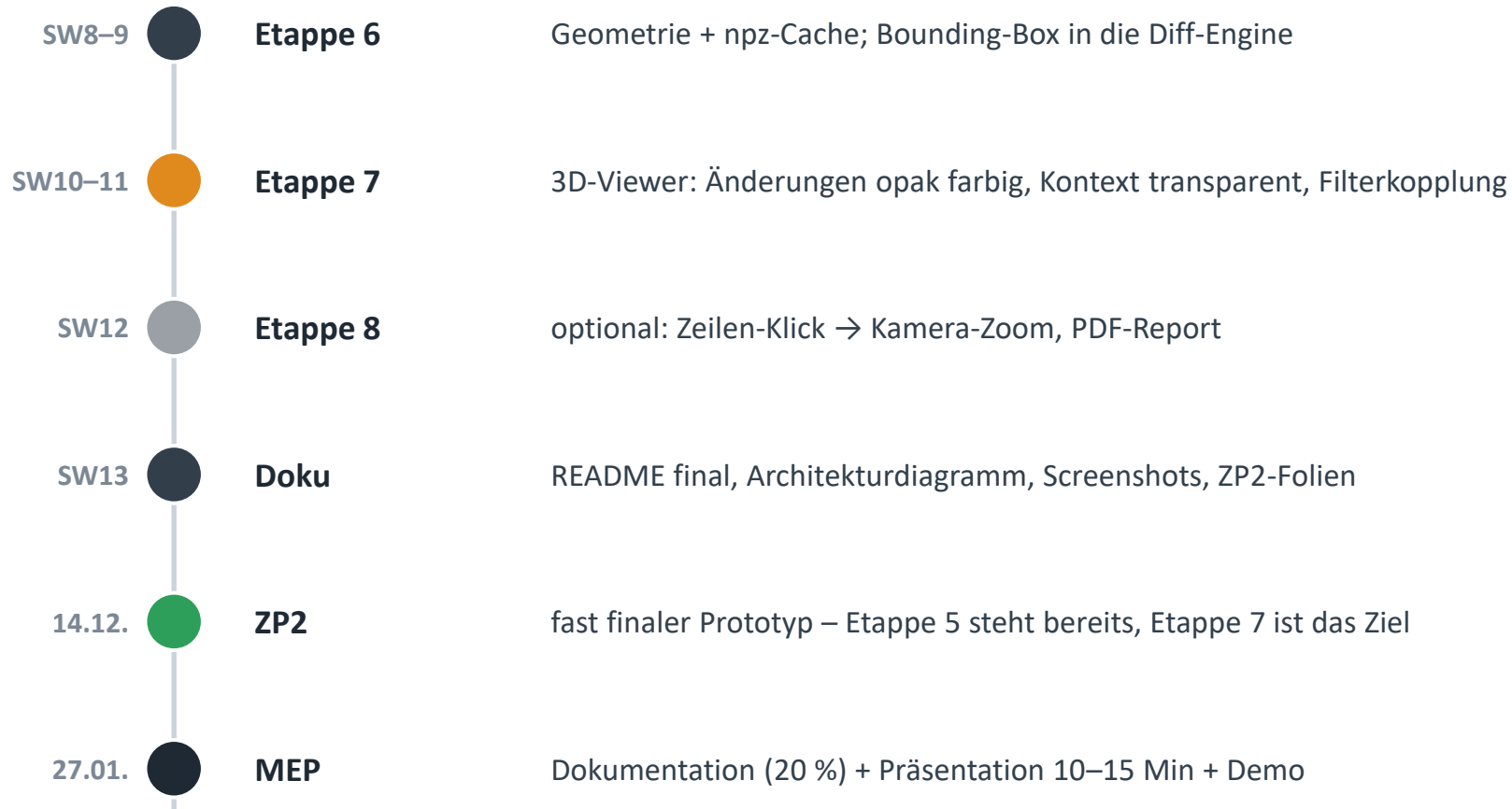
Code-Stil

Jede Zeile muss in der MEP erklärbar sein: drei Klassen, Funktionen unter 30 Zeilen, Docstrings, keine cleveren Einzeiler. ifcopenshell.api nur wo nötig, mit Begründung.

Zeitbudget

Ca. 40–45 h Entwicklung verfügbar. Etappe 5 ist der Abgabe-Meilenstein; Report und Screencast sind erste Streichkandidaten.

Nächste Schritte bis ZP2 (14.12.) und MEP



Frage an den Modulverantwortlichen: PyVista/stpyvista als Zusatzmodul für den 3D-Viewer in Ordnung? (IfcOpenShell erfüllt die Pflicht «mindestens ein externes Modul».)